

Ćwiczenie laboratoryjne

Porównanie algorytmów SA, PSO i GA na przykładzie problemu komiwojażera (TSP) dla zbioru berlin52

Wersja rozszerzona: opis zadania, pseudokody, fragmenty kodu Python, tabele wyników i pytania do analizy

1. Cel ćwiczenia

Celem ćwiczenia jest implementacja oraz porównanie trzech metaheurystycznych algorytmów optymalizacji: Simulated Annealing (SA), Particle Swarm Optimization (PSO) oraz algorytmu genetycznego (GA) dla problemu komiwojażera. Studenci pracują na klasycznym zbiorze TSPLIB berlin52, analizują jakość rozwiązań, czas działania oraz stabilność wyników.

2. Dane wejściowe i problem badawczy

Zbiór berlin52 zawiera współrzędne 52 miast. Zadaniem jest znalezienie trasy o minimalnej długości, która odwiedza każde miasto dokładnie raz i wraca do punktu startowego.

Element	Opis
Zbiór danych	berlin52.tsp (TSPLIB)
Liczba miast	52
Reprezentacja rozwiązania	Permutacja numerów miast
Funkcja celu	Suma długości odcinków w cyklu Hamiltona
Typ problemu	Optymalizacja kombinatoryczna

3. Zadania do wykonania

1. Pobrać zbiór berlin52 i wczytać współrzędne miast do programu w Pythonie.
2. Narysować położenie miast oraz przygotować funkcję obliczającą długość dowolnej trasy.
3. Zaimplementować algorytm Simulated Annealing dla TSP.
4. Zaimplementować algorytm genetyczny z operatorem krzyżowania i mutacji dla permutacji.
5. Zaimplementować wersję PSO dostosowaną do reprezentacji permutacyjnej albo przygotować uproszczoną adaptację opartą na ruchach swap/insert.
6. Uruchomić każdy algorytm co najmniej 10 razy i porównać uzyskane wyniki.
7. Zwizualizować najlepszą trasę na mapie miast oraz przedstawić tabelę wyników.

4. Pseudokody algorytmów

4.1. Simulated Annealing

```
Wejście: trasa początkowa  $x$ , temperatura początkowa  $T_0$ 
 $x\_best \leftarrow x$ 
 $T \leftarrow T_0$ 

dopóki  $T > T_{min}$ :
    dla  $i = 1$  do  $L$ :
         $x\_new \leftarrow \text{wygeneruj\_sąsiada}(x)$ 
         $\Delta \leftarrow \text{koszt}(x\_new) - \text{koszt}(x)$ 
        jeśli  $\Delta < 0$ :
             $x \leftarrow x\_new$ 
        w przeciwnym razie:
            zaakceptuj  $x\_new$  z prawdopodobieństwem  $\exp(-\Delta / T)$ 
        jeśli  $\text{koszt}(x) < \text{koszt}(x\_best)$ :
             $x\_best \leftarrow x$ 
     $T \leftarrow \alpha * T$ 

zwróć  $x\_best$ 
```

4.2. Algorytm genetyczny

```
Utwórz populację początkową  $P$ 
oceń wszystkie osobniki

dopóki warunek stopu nie jest spełniony:
    wybierz rodziców
    wykonaj krzyżowanie
    wykonaj mutację
    oceń nowe osobniki
    utwórz nową populację

zwróć najlepszy osobnik
```

4.3. Particle Swarm Optimization dla TSP

```
Utwórz populację cząstek (permutacje tras)
dla każdej cząstki zapamiętaj pbest
wyznacz gbest

dopóki warunek stopu nie jest spełniony:
    dla każdej cząstki:
        wykonaj sekwencję ruchów przybliżającą cząstkę do pbest i gbest
```

```
    oceń nową trasę  
    zaktualizuj pbest  
    zaktualizuj gbest
```

```
zwróć gbest
```

5. Fragmenty kodu Python

5.1. Wczytanie danych berlin52

```
def load_tsp(filename):  
    coords = []  
    with open(filename, "r", encoding="utf-8") as f:  
        read_coords = False  
        for line in f:  
            line = line.strip()  
            if line == "NODE_COORD_SECTION":  
                read_coords = True  
                continue  
            if line == "EOF":  
                break  
            if read_coords:  
                idx, x, y = line.split()  
                coords.append((float(x), float(y)))  
    return coords
```

5.2. Długość trasy

```
import math  
  
def route_length(route, coords):  
    total = 0.0  
    n = len(route)  
    for i in range(n):  
        a = coords[route[i]]  
        b = coords[route[(i + 1) % n]]  
        total += math.dist(a, b)  
    return total
```

5.3. Przykład sąsiedztwa dla SA

```
import random

def swap_neighbour(route):
    new_route = route[:]
    i, j = random.sample(range(len(route)), 2)
    new_route[i], new_route[j] = new_route[j], new_route[i]
    return new_route
```

6. Organizacja eksperymentu

Każdy algorytm należy uruchomić wielokrotnie, ponieważ metaheurystyki są wrażliwe na losowość. Zaleca się wykonać co najmniej 10 uruchomień dla tych samych parametrów i porównać wynik najlepszy, średni oraz najgorszy.

Algorytm	Najlepszy wynik	Średni wynik	Odchylenie std.	Czas [s]
SA				
PSO				
GA				

7. Wizualizacja trasy

Na końcu należy przedstawić wykres miast oraz połączyć je w kolejności wynikającej z najlepszej znalezionej permutacji. W sprawozdaniu warto umieścić oddzielne rysunki dla SA, PSO i GA oraz wskazać długość uzyskanej trasy.

```
import matplotlib.pyplot as plt

def plot_route(route, coords, title="Najlepsza trasa"):
    xs = [coords[i][0] for i in route] + [coords[route[0]][0]]
    ys = [coords[i][1] for i in route] + [coords[route[0]][1]]
    plt.figure(figsize=(8, 6))
    plt.plot(xs, ys, "-o", markersize=4)
    plt.title(title)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.grid(True, alpha=0.3)
    plt.show()
```

8. Pytania do analizy i wnioski

- Który algorytm uzyskał najlepszą trasę dla berlin52?
- Który algorytm był najbardziej stabilny między uruchomieniami?
- Jak parametry SA (T_0 , α , liczba iteracji) wpływają na wynik?
- Jakie trudności pojawiają się przy adaptacji PSO do problemu permutacyjnego?
- Który algorytm wybrać(a)byś do większych instancji TSP i dlaczego?